

Manual de Vue.js - Referencia Rápida basado en Gisquick Web

Basándome en el proyecto Gisquick Web, aquí tienes un manual de Vue.js con ejemplos reales del código:

1. Estructura Básica de un Componente Vue

```
<template>
  <!-- HTML template -->
  <div class="search-tool dark f-row-ac" :class="{expanded}">
    <v-btn class="toggle icon flat" @click="toggle">
      <v-icon name="magnifier"/>
    </v-btn>
  </div>
</template>

<script>
export default {
  name: 'SearchTool',
  components: { VAutocomplete, FeaturesViewer },
  props: {
    label: String
  },
  data() {
    return {
      expanded: false,
      loading: false
    }
  },
  computed: {
    // propiedades computadas
  },
  methods: {
    // métodos del componente
  }
}
</script>

<style lang="scss" scoped>
.search-tool {
  margin-top: 7px;
  background-color: #333;
}
</style>
```

2. Data y Reactividad

```
// En SearchTool.vue
data() {
  return {
    suggestions: [],
    feature: null,
    expanded: false,
    loading: false,
    error: '',
    result: null,
    selectedSearchType: 'normal',
    specificSearches: [],
    searchTypes: [
      { value: 'normal', text: this.$gettext('Cerca normal') }
    ]
  }
}
```

Concepto clave: Todo en `data()` es reactivo - cuando cambia, la UI se actualiza automáticamente.

3. Props - Comunicación Padre → Hijo

```
// En InfoPanel.vue
props: {
  selected: Object,
  layer: Object,
  features: Array,
  layers: Array,
  mode: {
    type: String,
    default: 'view'
  }
}
```

Uso en template padre:

```
<info-panel
  :features="features"
  :layer="layer"
  :mode.sync="mode"
  :selected="selection"
/>
```

4. Computed Properties

```
// En SearchTool.vue
computed: {
```

```

...mapState(['project']), // Vuex helpers

config() {
  return this.project.config.search ?? {}
},

service() {
  if (this.selectedSearchType === 'normal') {
    const name = 'barcelona'
    return this.barcelonaService()
  } else {
    return this.specificLayerSearch(this.selectedSearchType)
  }
},

features() {
  return this.feature ? [this.feature] : []
}
}

```

Concepto clave: Las computed properties se recalculan automáticamente cuando sus dependencias cambian.

5. Watchers - Observar Cambios

```

// En SearchTool.vue
watch: {
  project: {
    handler(newProject, oldProject) {
      console.log('📁 Project changed');
      if (newProject && newProject !== oldProject) {
        this.$nextTick(() => {
          this.initSpecificSearches();
        });
      }
    },
    deep: true, // observar cambios profundos
    immediate: true // ejecutar inmediatamente
  },

  selectedFeature(feature, oldFeature) {
    if (oldFeature) {
      // limpiar estado anterior
    }
    if (feature) {
      // configurar nuevo estado
    }
  }
}
}

```

6. Methods - Funciones del Componente

```
methods: {
  toggle() {
    this.expanded = !this.expanded
    if (this.expanded) {
      this.$nextTick(() => {
        this.$refs.autocomplete.focus()
      })
    } else {
      this.clear()
    }
  },

  clear() {
    this.feature = null
    this.result = null
    this.suggestions = []
  },

  async onInput(item) {
    if (this.result !== item) {
      this.feature = item ? Object.freeze(await this.service.getFeature(item)) :
null
      this.result = item
    }
    if (this.feature) {
      this.$map.ext.zoomToFeature(this.feature)
    }
  }
}
```

7. Lifecycle Hooks

```
// En SearchTool.vue
created() {
  console.log('🚀 Component created');
  this.initSpecificSearches()
},

mounted() {
  console.log('🚀 Component mounted');
  this.initThematicSearch();

  this.$nextTick(() => {
    this.initSpecificSearches();
  });
},

beforeDestroy() {
  // limpiar listeners, etc.
```

```
    unByKey(this.listener)
  }
```

Orden de ejecución:

1. **created** - componente creado, data disponible
2. **mounted** - componente montado en DOM
3. **beforeDestroy** - antes de destruir componente

8. Event Handling

```
<template>
  <!-- Click events -->
  <v-btn @click="toggle">Toggle</v-btn>
  <v-btn @click="$emit('close')">Close</v-btn>

  <!-- Input events -->
  <v-autocomplete
    @input="onInput"
    @text:update="onTextChangeDebounce"
    @clear="clear"
  />

  <!-- Custom events -->
  <info-panel
    @close="[mode = 'view', features = null]"
    @edit="onFeatureUpdate"
    @delete="onFeatureDeleted"
  />
</template>
```

9. Event Emission - Comunicación Hijo → Padre

```
// En componente hijo
methods: {
  closePanel() {
    this.$emit('close')
  },

  updateData(newData) {
    this.$emit('update', newData)
  },

  customEvent() {
    this.$emit('custom-event', {
      data: 'some data',
      timestamp: Date.now()
    })
  }
}
```

```
}  
}
```

10. Conditional Rendering

```
<template>  
  <!-- v-if: añade/quita del DOM -->  
  <div v-if="enabled" class="search-tool">  
    <!-- contenido -->  
  </div>  
  
  <!-- v-show: toggle visibility CSS -->  
  <div v-show="!collapsed" class="wrapper">  
    <!-- contenido -->  
  </div>  
  
  <!-- v-else -->  
  <div v-if="loading">  
    <v-spinner/>  
  </div>  
  <div v-else>  
    <!-- contenido principal -->  
  </div>  
  
  <!-- Múltiples condiciones -->  
  <template v-if="mode === 'edit'">  
    <!-- modo edición -->  
  </template>  
  <template v-else-if="mode === 'view'">  
    <!-- modo vista -->  
  </template>  
  <template v-else>  
    <!-- modo por defecto -->  
  </template>  
</template>
```

11. List Rendering

```
<template>  
  <!-- v-for básico -->  
  <div v-for="item in items" :key="item.id">  
    {{ item.name }}  
  </div>  
  
  <!-- v-for con index -->  
  <a  
    v-for="(item, index) in data"  
    :key="item.layer.name"  
    :class="{ active: layer === item.layer }"
```

```

    @click="setActiveLayer(item.layer)"
  >
    {{ item.layer.title }}
  </a>

  <!-- v-for en objeto -->
  <div v-for="(value, key) in object" :key="key">
    {{ key }}: {{ value }}
  </div>
</template>

```

12. Class y Style Binding

```

<template>
  <!-- Class binding -->
  <div class="search-tool" :class="{expanded, loading}">
    <div :class="['base-class', { active: isActive, disabled: !enabled }]">

  <!-- Style binding -->
  <div :style="{ height: height + 'px', transform: `rotate(${rotation}rad)` }">
    <div :style="heightStyle">

  <!-- En el script -->
  computed: {
    heightStyle() {
      const height = (this.minimized ? 1 : this.height) + 'px'
      return { height }
    }
  }
</template>

```

13. Template Refs

```

<template>
  <v-autocomplete ref="autocomplete" />
  <div ref="mapEl" class="map"/>
</template>

<script>
methods: {
  focusInput() {
    this.$refs.autocomplete.focus()
  },

  initMap() {
    // usar this.$refs.mapEl para acceder al elemento DOM
    const map = new Map({
      target: this.$refs.mapEl
    })
  }
}

```

```
}  
}  
</script>
```

14. Slots - Contenido Dinámico

```
<!-- Componente padre -->  
<template>  
  <tabs-header>  
    <template slot="tabs">  
      <a v-for="item in data" :key="item.layer.name">  
        {{ item.layer.title }}  
      </a>  
    </template>  
  </tabs-header>  
</template>  
  
<!-- Componente TabsHeader -->  
<template>  
  <div class="tabs-header">  
    <div class="tabs">  
      <slot name="tabs"/>  
    </div>  
  </div>  
</template>
```

15. Vuex - State Management

```
// En componente  
import { mapState } from 'vuex'  
  
export default {  
  computed: {  
    ...mapState(['project', 'user']),  
  
    // equivale a:  
    // project() { return this.$store.state.project },  
    // user() { return this.$store.state.user }  
  },  
  
  methods: {  
    updateTool() {  
      // Commit mutation  
      this.$store.commit('activeTool', 'search')  
  
      // Dispatch action  
      this.$store.dispatch('loadProject', projectName)  
    }  
  }  
}
```



```
}  
}
```

16. Async/Await

```
// En SearchTool.vue  
methods: {  
  async suggest(text) {  
    return await this.service.autocomplete(text)  
  },  
  
  async onInput(item) {  
    if (this.result !== item) {  
      this.feature = item ? Object.freeze(await this.service.getFeature(item)) :  
null  
      this.result = item  
    }  
  },  
  
  async fetchData() {  
    try {  
      this.loading = true  
      const response = await this.$http.get('/api/data')  
      this.data = response.data  
    } catch (error) {  
      console.error('Error:', error)  
      this.error = error.message  
    } finally {  
      this.loading = false  
    }  
  }  
}
```

17. Mixins - Código Reutilizable

```
// En Map.vue y MobileMap.vue  
import Map from '@mixins/Map'  
  
export default {  
  mixins: [Map], // incluye todas las propiedades del mixin  
  
  // el componente puede sobrescribir propiedades del mixin  
  mounted() {  
    // código específico del componente  
    this.updateMapSize()  
    // código del mixin también se ejecuta  
  }  
}
```

18. Debouncing

```
import debounce from 'lodash/debounce'

export default {
  methods: {
    // Versión con función externa
    onTextChangedDebounced: debounce(async function (text) {
      this.onTextChanged(text)
    }, 400),

    // Uso en template
    // @text:update="onTextChangedDebounced"
  }
}
```

19. Portal/Teleport (Vue 2 usa portal-vue)

```
<template>
  <!-- Renderizar contenido en otro lugar del DOM -->
  <portal to="main-panel">
    <div class="edit-panel light f-col" key="edit">
      <!-- contenido del panel -->
    </div>
  </portal>

  <!-- Target donde se renderiza -->
  <portal-target name="main-panel"/>
</template>
```

20. Transiciones

```
<template>
  <!-- Transición simple -->
  <transition name="fade">
    <div v-if="mapLoading" class="status">
      <v-spinner/>
    </div>
  </transition>

  <!-- Transición de lista -->
  <transition name="slide-top-transition">
    <portal-target name="right-panel"/>
  </transition>
</template>
```

```
<style>
.fade-enter-active, .fade-leave-active {
  transition: opacity 0.5s;
}
.fade-enter, .fade-leave-to {
  opacity: 0;
}
</style>
```

Consejos Importantes del Proyecto:

1. Usa **this.\$nextTick()** cuando necesites esperar a que Vue actualice el DOM
2. Congela objetos con **Object.freeze()** para optimizar rendimiento en listas grandes
3. Usa **computed properties** en lugar de methods para valores derivados
4. Maneja errores siempre con try/catch en operaciones async
5. Usa **keys únicas** en v-for para mejor rendimiento
6. Limpia listeners en beforeDestroy para evitar memory leaks

Este manual cubre los conceptos fundamentales de Vue.js tal como se usan en el proyecto Gisquick Web.

Similar code found with 2 license types